



Informe Proyecto 1

Departamento de Ingeniería de Sistemas y Computación
ISIS-1226 Diseño y Programación Orientada a Objetos

Juan David Vasquez Leyva - 202426062

David Hortua - 202510382

Juan Felipe Leon - 202510397

Universidad de los Andes

1. El problema que se resuelve

El proyecto tiene como propósito resolver el problema de El Board Game Cafe “Dulces n Dados” el cual es un establecimiento que combina una cafetería que, a su vez, funciona como un lugar para poder jugar juegos de mesa. El negocio requiere un sistema de información que permita gestionar de manera eficiente sus operaciones, abarcando tres grandes áreas: la gestión del inventario de juegos de mesa (préstamo y venta), el funcionamiento del café (empleados, menú y facturas) y las tareas administrativas. Este documento presenta el modelo, los requerimientos funcionales y las restricciones del sistema.

2. Análisis

A partir del análisis del caso de estudio se llegó a los siguientes elementos. El sistema de “Dulces n Dados” se divide en tres grandes categorías: el servicio de préstamo de juegos y reserva de mesas, el servicio de cafetería y restaurante, y el rol del administrador del local. Además, el sistema también cuenta con tres tipos de usuario: Cliente, Empleado (Cocinero o Mesero) y Administrador, cada uno con su login y contraseña.

Con respecto al servicio de préstamo de juegos, para hacer un préstamo de un juego el cliente debe tener una mesa reservada, en la que se especifique el número de personas y si hay menores de edad, verificando que no pase la capacidad máxima del negocio. Existen tres restricciones adicionales según el tipo de juego: los juegos marcados como difíciles necesitan que haya un mesero capacitado para enseñar las reglas. Además, el café maneja un inventario de juegos a la venta, los cuales son distintos a los que están para préstamo, y tanto los clientes como los empleados pueden tenerlos, estos últimos tienen un código de descuento propio.

En cuanto a la cafetería, el menú incluye productos de pastelería y bebida, que pueden ser alcohólicas o calientes. La venta de algunos productos no está permitida dependiendo de los integrantes de la mesa. Por ejemplo, si en la mesa hay niños, entonces no se puede entregar bebidas alcohólicas. También, las bebidas calientes no pueden servirse cuando se está jugando un juego de acción. Todas las ventas tienen impuestos y además le dan punto de fidelidad al cliente y esto puede usarse como descuentos en compras.

Es importante mencionar que los empleados tienen turnos específicos que se repiten cada semana. El sistema debe garantizar que siempre haya al menos 1 cocinero y 2 meseros al mismo tiempo.

Por último, el administrador tiene acceso a la información de todos los juegos que hay en los inventarios de préstamo y de venta. Puede hacer pedidos de juegos nuevos para ambos

inventarios, o reparar juegos. También, tiene un historial completo de todos los préstamos del local y su información. Además, tiene que hacer un informe con todos los datos de los juegos y café, incluyendo fechas, horas, montos, etc.

3. Modelo

El modelo se construyó identificando las entidades principales, sus atributos y las relaciones entre ellas. A continuación se presenta una descripción de cada clase y sus características. En esto nos basamos para hacer el UML.

3.1 Usuario

La clase Usuario es abstracta y tiene las siguientes subclases.

Usuario (abstract) : login : String, contraseña : String.

Cliente : idCliente : String, juegosComprados : List<JuegoDeMesa>, juegosFavoritos : List<JuegoDeMesa>, juegosReservados (maximo 2), puntosDeFidelidad : float, mesa : Mesa.

Administrador : Hereda usuario y no tiene más atributos propios.

Empleado (abstract) : turnoLaboral : List<String>, codigoDescuento : String, juegosFavoritos : List<JuegosDeMesa>. Está clase va a ser la superclase de Cocinero y Mesero.

Cocinero (hereda Empleado) : Hereda empleado y tiene mínimo un turno.

Mesero (hereda Empleado) : juegosDificiles : List<JuegoDeMesa>. Debe tener mínimo 2 turnos.

3.2 Tienda de Juegos

Gestiona el inventario de juegos para préstamo y venta.

TiendaDeJuegos : facturas : List<Factura>, inventarioJuegos : InventarioJuegos

InventarioJuegos : juegosPrestamo : List<JuegosDeMesa>, juegosVenta : List<JuegosDeMesaVenta>, historialPrestamos : List<Prestamo>

JuegoDeMesa : idJuego : String, enUso : boolean, añoPublicacion : datetime, empresaProduccion : String, tipoDeJuego : String, minJugadores : int, maxJugadores : int, edadMinima : int, categoria (Cartas / Tablero / Acción) : String, estado : String, dificil : boolean.

Prestamo : idJuego : String, idMesa : String, fechaInicio : datetime, fechaFin : datetime, cliente : String.

3.3 Cafe

Gestiona el funcionamiento de la cafetería.

Cafe : mesa : List<Mesa>, capacidadMaxClientes : int, empleados : List<Empleado>, menuProductos : List<Producto>, facturas : List <Factura>

Mesa : idMesa : String, cantidadClientes : int, tieneNinos : boolean, tieneJovenes : boolean, cliente : Cliente.

Turno : diaSemana : datetime, empleado : Empleado.

Producto (abstract) : nombre : String, precio : float. Esta clase es la superclase de Bebida y Pasteleria.

Bebida (hereda Producto) : alcoholica : boolean, caliente : boolean.

Pasteleria (hereda Producto) : alergenos : List<String>

3.4 Factura

Gestiona los registros de las ventas.

Factura : fecha : datetime, subtotal : float, impuestos : float, propina : float, detalles : List<DetalleVenta>

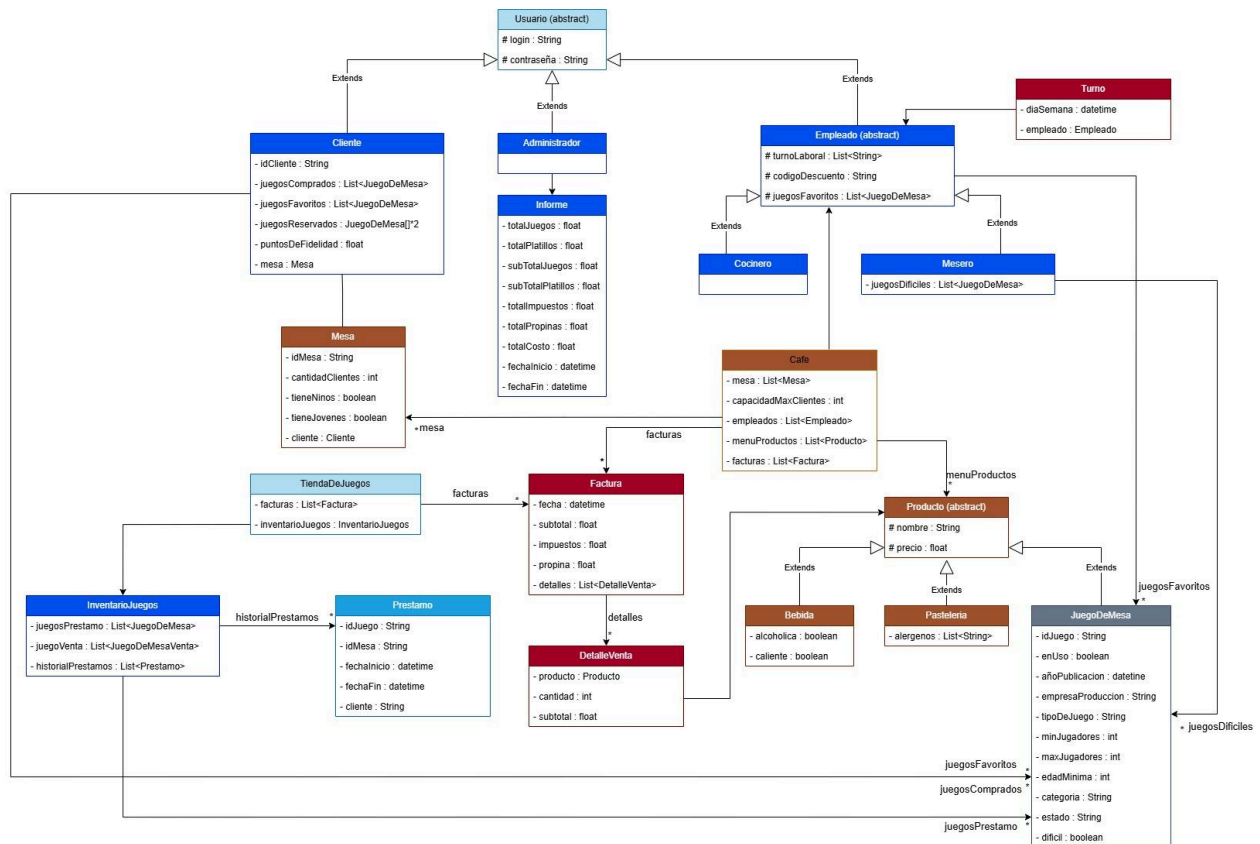
DetalleVenta : producto : Producto, cantidad : int, subtotal : float.

3.5 Admin e Informe

La clase Administrador debe hacer un informe que debe contener lo siguiente.

Informe : totalJuegos : float, totalPlatillos : float, subTotalJuegos : float, subTotalPlatillos : float, totalImpuestos : float, totalPropinas : float, totalCosto : float, fechaInicio : datetime, fechaFin : datetime.

4. Diagrama de clases



En este diagrama de clases en UML se ve representado el sistema de “Dulces n Dados”. Se pueden apreciar las clases de Empleado, Administrador, y Cliente, las cuales heredan a Usuario. Esto se debe a que todos los usuarios sin importar su tipo, requieren login y contraseña, y tenerlos como una extensión de Usuario evita la repetición de código.

La clase Empleado es abstracta y tiene como subclases a Mesero y Cocinero. El mesero debe conocer juegos difíciles, mientras que el cocinero es un empleado común y corriente. Por su parte, el Administrador tiene un historial completo de los préstamos realizados en el local, junto con su información, además, debe hacer informes detallados de las ventas.

El cliente se relaciona con la mesa a través de la reserva y con los juegos de mesas por medio de la compra o pedirlos prestados. Además, al tener una mesa, puede hacer uso de las funciones de la cafetería, como ordenar productos tanto de pastelería como bebidas. Es importante mencionar que, tanto las bebidas, los productos de pastelería y los juegos de mesa son herederos de la clase Producto. Cada vez que se hace una compra, al cliente se le genera una factura por cada compra de juegos y cada gasto en el cafe.

Finalmente, es importante mencionar que los inventarios de préstamo y de venta se separaron dentro de la clase `InventarioJuegos` para mantener la distinción entre ambos, ya que el texto menciona que ambas tienen listas completamente diferentes.

5. Requerimientos Funcionales

A continuación se presentan los requerimientos funcionales del sistema, organizados por el actor principal que los ejecuta. Para cada requerimiento se describe su propósito, las entradas que recibe, la salida que produce y las excepciones que puede lanzar.

Lista de excepciones

El sistema utiliza las siguientes excepciones personalizadas para señalar condiciones de error de forma precisa:

- **EntidadNoEncontradaException** : Se lanza cuando un objeto buscado por ID no existe en el sistema.
- **CapacidadExcedidaException** : Se lanza cuando se intenta agregar más personas de las que permite la capacidad del café.
- **JuegoNoDisponibleException** : Se lanza cuando se intenta prestar un juego que ya está en uso.
- **RestriccionEdadException** : Se lanza cuando el juego solicitado no es apto para la edad de los comensales de la mesa.
- **RestriccionJugadoresException** : Se lanza cuando el número de personas en la mesa está fuera del rango de jugadores del juego.
- **ConflictoHorarioException** : Se lanza cuando la asignación o cambio de turno viola el mínimo de empleados requerido.
- **OperacionNoPermitidaException** : Se lanza cuando un actor intenta realizar una acción para la que no tiene permisos.
- **InventarioInsuficienteException** : Se lanza cuando no hay suficientes unidades de un juego en el inventario.
- **SolicitudNoValidaException** : Se lanza cuando una solicitud de cambio de turno o de informe tiene parámetros inválidos.
- **EstadoInvalidoException** : Se lanza cuando se intenta realizar una operación sobre un objeto que no está en el estado adecuado.

- **Requerimientos del Cliente**

reservarJuego

Actor: Cliente

Descripción: Permite a un cliente solicitar el préstamo de un juego de mesa para su mesa. El sistema verifica que el juego esté disponible, que sea apto para el número de personas y las edades presentes en la mesa, y que la capacidad del café no se haya superado. Retorna un objeto Prestamo que registra el inicio del préstamo.

Entradas:

- **clientId:** String
- **juegoId:** String
- **mesaId:** String

Salida:

- Prestamo

Excepciones:

- JuegoNoDisponibleException
- RestriccionEdadException
- RestriccionJugadoresException
- CapacidadExcedidaException

solicitarJuegoPrestado

Actor: Cliente

Descripción: Variante de reservarJuego que también permite a un cliente solicitar un juego prestado para su mesa. Al igual que reservarJuego, valida disponibilidad, restricciones de edad y de número de jugadores antes de registrar el préstamo.

Entradas:

- **clientId:** String
- **mesaId:** String
- **juegoId:** String

Salida:

- Prestamo

Excepciones:

- JuegoNoDisponibleException
- RestriccionEdadException
- RestriccionJugadoresException

comprarJuegoCliente

Actor: Cliente

Descripción: Permite a un cliente comprar uno o varios juegos del inventario de venta en una sola transacción. El sistema verifica que haya suficientes unidades disponibles, aplica el IVA del 19% y genera una factura detallada. Si el cliente tiene un código de descuento válido, este se aplica antes del cálculo de impuestos.

Entradas:

- **clienteId:** String
- **listaJuegos:** List<JuegoDeMesa>

Salida:

- Factura

Excepciones:

- InventarioInsuficienteException

agregarPuntosDeFidelidad

Actor: Cliente (automático tras cada compra)

Descripción: Calcula y acumula los puntos de fidelidad del cliente tras cada compra. El sistema agrega el 1% del monto de la compra como puntos, donde cada punto equivale a un peso y puede ser usado como descuento en compras futuras.

Entradas:

- **clienteId:** String
- **montoCompra:** float

Salida:

- float (puntos agregados)

Excepciones:

- EntidadNoEncontradaException

- **Requerimientos del Empleado**

solicitarCambioHorario

Actor: Empleado

Descripción: Permite a un empleado solicitar el cambio de uno de sus turnos semanales a otro día disponible. El sistema verifica que el cambio no deje al café con menos del mínimo requerido de cocineros (1) o meseros (2) en ningún turno afectado. La solicitud queda en estado pendiente hasta que el administrador la apruebe o rechace.

Entradas:

- **empleadold:** String
- **turnoActual:** Turno
- **turnoNuevo:** Turno

Salida:

- SolicitudCambio

Excepciones:

- SolicitudNoValidaException

solicitarIntercambioHorario

Actor: Empleado

Descripción: Permite a dos empleados intercambiar sus turnos entre sí. El sistema verifica que el intercambio no viole los mínimos de cobertura del café en ninguno de los días involucrados. La solicitud queda pendiente hasta la aprobación del administrador.

Entradas:

- **empleado1Id:** String
- **empleado2Id:** String
- **turno1:** Turno
- **turno2:** Turno

Salida:

- SolicitudCambio

Excepciones:

- SolicitudNoValidaException

sugerirPlatillo

Actor: Empleado

Descripción: Permite a cualquier empleado (cocinero o mesero) proponer un nuevo producto para el menú del café. La sugerencia queda registrada en el sistema a la espera de que el administrador la evalúe y decida si la incorpora al menú.

Entradas:

- **empleadoid:** String
- **producto:** Producto

Salida:

- void

Excepciones:

- OperacionNoPermitidaException

- **Requerimientos del Administrador**

repararJuego

Actor: Administrador

Descripción: Permite al administrador registrar la reparación de un juego del inventario de préstamo. El estado del juego debe ser distinto de 'Nuevo' para poder repararse. Internamente, se reemplaza la copia dañada por una del inventario de ventas y se actualiza el estado del juego.

Entradas:

- **juegoid:** String

Salida:

- void

Excepciones:

- EstadoInvalidoException
- EntidadNoEncontradaException

darJuegoPorRobado

Actor: Administrador

Descripción: Permite al administrador marcar un juego como desaparecido cuando no ha sido devuelto tras un período razonable. El juego se elimina del inventario activo y su estado se actualiza a 'Robado', quedando registrado en el historial de préstamos.

Entradas:

- **juegold:** String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

venderJuego

Actor: Administrador

Descripción: Permite al administrador registrar la venta directa de un juego a un cliente desde el inventario de venta. El sistema verifica que haya stock disponible, aplica los impuestos correspondientes y genera una factura.

Entradas:

- **clienteld:** String
- **juegold:** String
- **cantidad:** int

Salida:

- Factura

Excepciones:

- InventarioInsuficienteException
- EntidadNoEncontradaException

asignarHorario

Actor: Administrador

Descripción: Permite al administrador asignar un nuevo turno a un empleado. El sistema verifica que la asignación no genere conflictos de cobertura mínima del café antes de confirmarla.

Entradas:

- **empleadoid:** String
- **turno:** Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

asignarTurnoHorario

Actor: Administrador

Descripción: Variante de asignarHorario que permite al administrador establecer el turno semanal de un empleado específico. Valida las mismas restricciones de cobertura mínima.

Entradas:

- **empleadoid:** String
- **turno:** Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

modificarTurnoHorario

Actor: Administrador

Descripción: Permite al administrador modificar el turno existente de un empleado, reemplazando el turno antiguo por uno nuevo. El sistema valida que el cambio no deje al café sin la cobertura mínima requerida en ninguno de los días involucrados.

Entradas:

- **empleadold:** String
- **turnoAntiguo:** Turno
- **turnoNuevo:** Turno

Salida:

- void

Excepciones:

- ConflictoHorarioException

aprobarSolicitudHorario

Actor: Administrador

Descripción: Permite al administrador revisar y aprobar una solicitud de cambio o intercambio de turno previamente enviada por un empleado. Una vez aprobada, el sistema actualiza los turnos correspondientes y notifica a los empleados involucrados.

Entradas:

- **administradorId:** String
- **solicitudId:** String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

aceptarSugerenciaPlatillo

Actor: Administrador

Descripción: Permite al administrador revisar una sugerencia de platillo enviada por un empleado y aprobarla para que sea incorporada al menú del café.

Entradas:

- **administradorId:** String
- **producto:** Producto

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

agregarPlatillo

Actor: Administrador

Descripción: Permite al administrador agregar directamente un nuevo producto al menú del café, sin necesidad de que provenga de una sugerencia. El producto debe tener un estado válido para poder ser incorporado.

Entradas:

- **producto:** Producto

Salida:

- void

Excepciones:

- EstadoInvalidoException

moverJuegoDelInventarioAVentas

Actor: Administrador

Descripción: Permite al administrador mover un juego del inventario de préstamo al inventario de ventas. El sistema verifica que haya unidades suficientes en el inventario de préstamo antes de realizar el movimiento.

Entradas:

- **juegoid:** String

Salida:

- void

Excepciones:

- `InventarioInsuficienteException`

reabastecerInventarioVentas

Actor: Administrador

Descripción: Permite al administrador comprar y agregar nuevas unidades de un juego al inventario de ventas. El juego debe estar en un estado válido para poder ser reabastecido.

Entradas:

- **juegold:** String
- **cantidad:** int

Salida:

- void

Excepciones:

- `EstadoInvalidoException`

reabastecerInventarioPrestamo

Actor: Administrador

Descripción: Permite al administrador comprar y agregar nuevas unidades de un juego al inventario de préstamo. Al igual que con el inventario de ventas, el juego debe estar en un estado válido.

Entradas:

- **juegold:** String
- **cantidad:** int

Salida:

- void

Excepciones:

- `EstadoInvalidoException`

reportarJuegoRobado

Actor: Administrador

Descripción: Permite al administrador reportar formalmente un juego como robado. El juego se marca como desaparecido en su estado y queda registrado en el historial de préstamos para efectos contables e históricos.

Entradas:

- **juegold:** String

Salida:

- void

Excepciones:

- EntidadNoEncontradaException

● Requerimientos de consulta (Administrador)

consultarEstadoJuego

Actor: Administrador

Descripción: Permite al administrador consultar el estado actual de un juego específico del inventario (Nuevo, Bueno, Falta una pieza, Robado, etc.). Esta información es exclusiva del administrador y no es visible para los clientes.

Entradas:

- **juegold:** String

Salida:

- String (estado actual del juego)

Excepciones:

- EntidadNoEncontradaException

consultarNumPrestamosJuego

Actor: Administrador

Descripción: Permite al administrador consultar cuántas veces ha sido prestado un juego específico a lo largo de toda la historia del café. Es útil para identificar los juegos más populares y tomar decisiones de inventario.

Entradas:

- **juegold:** String

Salida:

- int (número total de préstamos)

Excepciones:

- EntidadNoEncontradaException

consultarFechaPrestamoJuego

Actor: Administrador

Descripción: Permite al administrador consultar todas las fechas en las que un juego específico ha sido prestado. Retorna una lista de fechas para tener el historial completo de uso del juego.

Entradas:

- **juegold:** String

Salida:

- List<datetime> (fechas de préstamo)

Excepciones:

- EntidadNoEncontradaException

generarInforme

Actor: Administrador

Descripción: Permite al administrador generar un informe financiero detallado del café para un período de tiempo determinado. El informe incluye el total de ventas de juegos y de cafetería, separados por subtotal, impuestos y propinas, con granularidad diaria, semanal o mensual según el rango de fechas indicado.

Entradas:

- **fechaInicio:** datetime
- **fechaFin:** datetime

Salida:

- Informe

Excepciones:

- SolicitudNoValidaException